一、這一週在學什麼

第八週我們看過模型怎麼給出框、類別和分數，也自己在標註練習台上當了一次標註員。這一週接著問三件事。標好的照片，怎麼變成一個會認東西的模型（訓練）？練好的模型怎麼用，用得好不好怎麼算（推論與打分數）？車子一秒鐘拍十幾張照片，怎麼知道這一張裡的行人，就是上一張的那一個（追蹤）？

三節都用同一個例子：三角錐（越南文 cọc tiêu）。所有的動畫和數字，都來自老師在一台 MacBook 上真的訓練的一次：30 張照片、50 輪、160 秒。不是畫出來的，只有標明「示意」的圖例外。

節次	主題	要帶走的一句話
第 1 節	從標註到訓練	照片分三組；借練好的模型來教；練過頭就是背答案，留模擬考最好的一輪。
第 2 節	推論與打分數	框對、多框、漏掉；精確度、召回率；門檻愈高，多框愈少、漏掉愈多。
第 3 節	追蹤	每個東西一個號碼；有了號碼才算得出速度；被擋住、被框兩次，號碼就會換。

學習目標

讀完這份講義，你應該可以：

1. 說出訓練、推論、評估各是什麼，以及一個模型從標註到打分數要走哪七步。
2. 說明為什麼照片要分成訓練、驗證、測試三組，而且要整段影片一起分。
3. 用自己的話說明遷移學習和微調，以及練過頭（過度擬合）是什麼、怎麼挑出最好的一輪。
4. 說出框對、多框、漏掉，算出精確度和召回率，並說明門檻拉高、拉低時兩個分數怎麼變。
5. 說明追蹤怎麼用 **IoU** 配對、怎麼算速度，以及追蹤什麼時候會出錯。

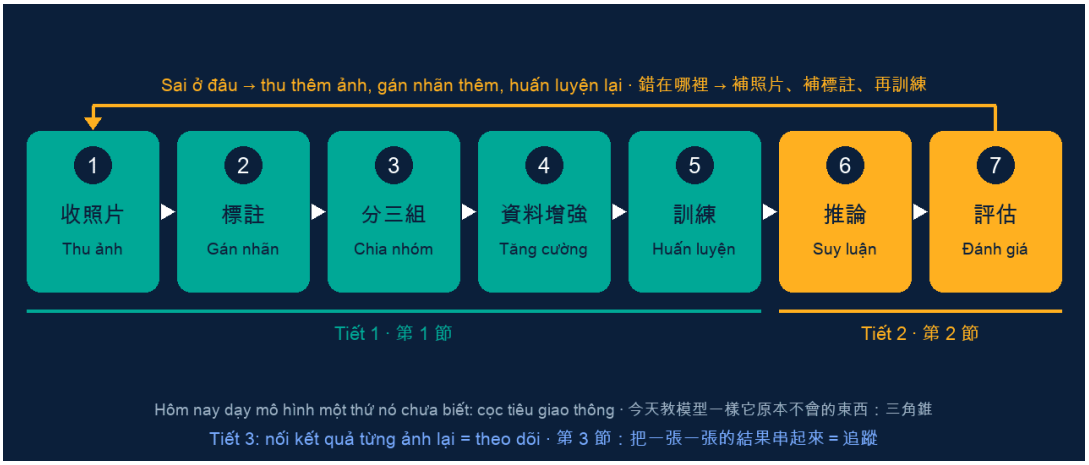
先記三個詞

模型：一個學會認東西的程式，可以想成一個正在學認東西的小朋友。訓練（training）：教這個小朋友，給他看很多照片、告訴他正確答案，像上課加寫練習題。推論（inference）：訓練完了，拿一張他沒看過的照片，請他自己說裡面有什麼，像上場考試。

二、訓練：從標註到一個會認三角錐的模型（第 1 節）

整條流程：七步，走完還要回頭

教電腦認東西，跟教小朋友認東西很像。下面七步，是一個模型從照片到打分數要走的路。第 1 節講前五步，第 2 節講後兩步，第 3 節把一張一張的結果串起來（追蹤）。

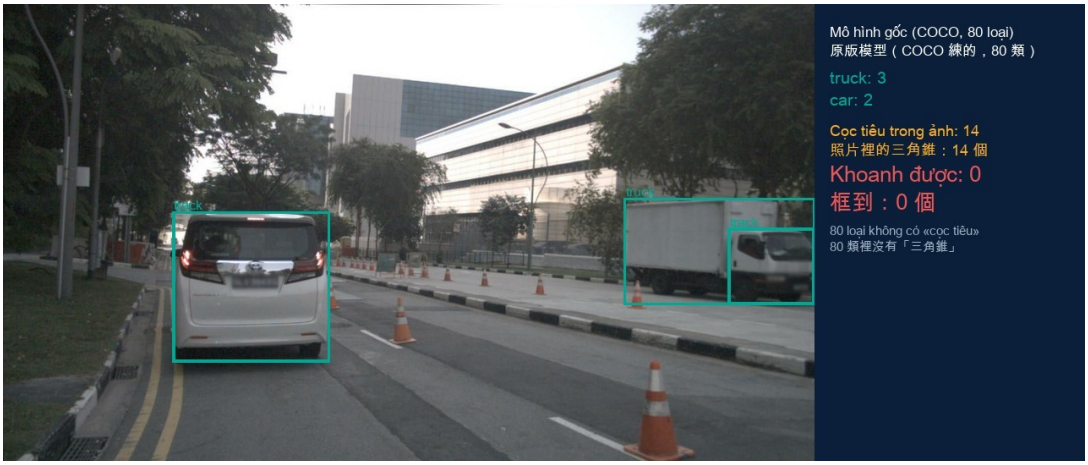


七步流程。評估完找出錯的地方，再回到第一步（上面那條箭頭）。

步	名稱	做什麼	像什麼
1	收照片	找很多有三角錐的照片	準備課本
2	標註	每張照片上，把三角錐框起來，告訴模型「這個就是」	拿螢光筆圈出來
3	分三組	練習題、模擬考、期中考	留一些題目不給看
4	資料增強	同一張照片翻過來、調暗、放大，變出很多種樣子	同一個觀念，換很多種題目
5	訓練	一輪一輪：猜、對答案、改一點點	寫練習題
6	推論	拿沒看過的照片，請模型自己框	上場考試
7	評估	跟標準答案比，算框對、多框、漏掉	改考卷

評估完，一定會找到模型錯的地方：某一種照片它老是認錯。那就回到第一步，多收那一種照片、標好、再練一次。這就像考完試，把錯的題目抄進錯題本，針對弱的地方再練。這個循環，業界叫資料引擎（data engine），第 2 節最後會再講。

為什麼選三角錐：現成的模型沒學過



新加坡一段施工路段（**scene-0061 #23**）。原版模型框到卡車 **3** 台、汽車 **2** 台，照片裡 **14** 個三角錐一個都沒框到。

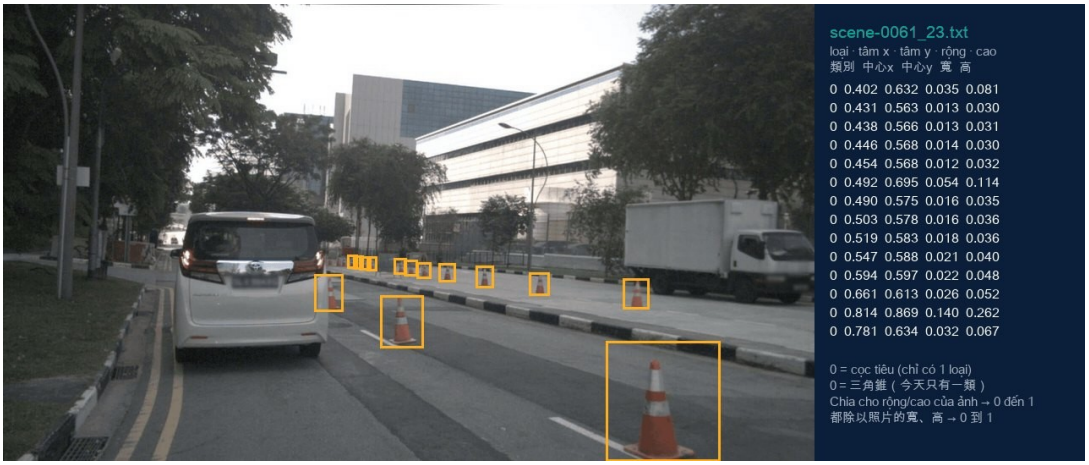
模型：**Ultralytics YOLO11n**，**COCO** 預訓練、分數門檻 **0.25**。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

這次用的模型叫 **YOLO**，是 You Only Look Once 的縮寫，意思是「只看一次」：整張照片看一眼，就把所有東西的位置一次框出來。版本是 **YOLO11n**（n 是最小的一種，跑得最快），由 Ultralytics 公開，可以免費下載。

它已經用一套叫 **COCO** 的資料練過。COCO 可以想成一本很厚的習題本：大約 11.8 萬張照片，每一張都有人先標好答案，教了 80 種東西：人、汽車、卡車、公車、機車、紅綠燈、停車標誌……可是這 80 種裡面，沒有三角錐。不是它眼睛不好，是從來沒人教過它。

三角錐對自駕車很重要：它告訴車子前面在施工、這個車道封起來了，不能開進去。所以今天的任務是：教這個模型認得三角錐。自己訓練最有用的時候，就是教它原本不會的東西。

標註：一個三角錐一個框，一個框存成一行



scene-0061 #23：**14** 個三角錐，一個一個畫上框。右邊是存檔的文字檔（**YOLO** 格式），一個框一行。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

標註就像拿一支螢光筆，在照片上把三角錐圈出來，告訴模型「這個就是」。標註的人從三角錐的左上角拉到右下角，畫一個框；畫好一個，檔案就多一行。這張照片有 14 個三角錐，所以是 14 行。遠的三角錐很小，只有幾個像素，也要框（第八週講過 nuScenes 的規則：感測器看得到，就要標）。

這個檔案的格式叫 **YOLO 格式**，和第八週標註練習台「下載標註」的一樣。每一行有五個數字：類別（三角錐是 0），加上框的中心在哪裡、框多寬多高。就像一張很短的地址：先說這是什麼東西，再說它在照片的哪裡、有多大。這五個數字怎麼算，放在最後的補充。

這次訓練用的資料是 **30 張照片、一共 282 個三角錐**。這些框是 nuScenes 請人標好的，不是我們畫的。282 個框，如果是你一個一個畫，要畫多久？真正的資料集是幾萬張照片、幾百萬個框。

照片要分三組：練習題、模擬考、期中考



訓練組 **30** 張（新加坡 **scene-0061**）；驗證組 **8** 張（波士頓 **scene-0655** 前段）；測試組 **17** 張（同一段後段，訓練時完全沒看過）。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

表 **1**　三組照片

組別	比喻	這次用的照片	用途
訓練組 training set	練習題	30 張，新加坡施工路段，282 個三角錐	模型從這裡學；只有這一組能改模型的參數
驗證組 validation set	模擬考	8 張，波士頓，13 個三角錐	每練完一輪考一次；人看成績，決定留哪一輪；不拿來學
測試組 test set	期中考	17 張，波士頓，90 個三角錐	訓練時完全不碰；最後才打開，只考一次

訓練組是練習題。模型裡有 260 萬個可以調整的數字，叫**參數**，只有練習題能改它們。

驗證組是模擬考。每練完一輪，就拿它考一次，看模型現在的程度。模擬考考幾分，不會回頭去改那 260 萬個參數，也不會把模擬考的照片放進練習題。那成績有什麼用？給我們人看的：我們看成績，決定留哪一輪。是人在做決定，模型不會因為考了模擬考而改變。考卷我們可以看、可以檢討，但不能把答案教給模型，不然它就變成練習題了。

測試組是期中考。訓練的時候，模型一張都沒看過，最後才拿出來考一次。為什麼期中考的題目不能先給大家看？因為看過了，分數就不能代表你真的會。驗證組不一樣：它一輪一輪反覆拿來考，我們也一直根據它的成績做決定，所以它的成績會偏好看。測試組沒有參與任何決定，才是誠實的成績。

還有兩點要老實說。第一，練習題在新加坡，考試在波士頓：兩個城市的路、天氣、三角錐的擺法都不一樣，這樣考，才知道模型換了地方還行不行。第二，這次的驗證組和測試組，是波士頓同一段影片的前後兩半，中間只隔 0.5 秒，兩組太像，所以測試成績可能比真正的程度稍微好看一點。

> **名字對調的陷阱**
>
> 有的書會把「驗證組」和「測試組」的名字對調來用。看別人的資料時，先問清楚：哪一組是拿來選輪次的，哪一組是最後只考一次的。這門課的說法：驗證組 = 模擬考（訓練時每輪考一次），測試組 = 期中考（最後只考一次）。

分組的陷阱：相鄰兩張照片幾乎一樣



scene-0061 的第 21 個畫面（左）和第 22 個畫面（右），只差 0.5 秒。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

nuScenes 每 0.5 秒標一次，所以相鄰兩張只差半秒：車子往前開了幾公尺，三角錐還是那些三角錐，幾乎一模一樣，像翻頁動畫書的相鄰兩頁。

假設把這段影片的照片打散，隨便抽幾張當練習題，剩下的當考題：第 21 個畫面在練習題，第 22 個畫面在考卷上。模型考第 22 個畫面，其實是在考它練習過的題目。成績當然很好，可是那不是真的會，只是看過。這個問題叫資料洩漏（data leakage）：考題的內容事先漏進了練習題，成績就不準了。就像老師把練習題改一個數字就拿來考，大家都考得很好，換一題真的新的就不會了。

所以做法是：整段影片分在同一組。新加坡那一整段是練習題，波士頓那一段拿來考。真正的研究和公司也這樣做，nuScenes 官方提供的訓練組、測試組，也是整段影片分開的。

遷移學習：不從零開始，借一個已經看過 11.8 萬張照片的模型

課本準備好了，開始訓練。先回答一個問題：只有 30 張照片，夠嗎？我們試了兩種起點，用同樣的 30 張照片、同樣的設定，都練 50 輪，再拿 17 張測試照片（90 個三角錐）來考。



上排：還沒教三角錐，把同一張波士頓照片送進去，看模型第一層看到什麼（左：從零開始；右：**COCO** 練好的原版）。下排：同樣 **30** 張、練 **50** 輪之後，考這張照片（左：從零開始，**8** 個只框對 **1** 個；右：從 **COCO** 開始，**8** 個全對）。白色虛線 = 標準答案，綠 = 框對，紅 = 多框。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

表 **2**　兩種起點（同樣 **30** 張照片、**50** 輪、**17** 張測試照片 **90** 個三角錐）

起點	框對（**Đúng**）	多框（**Thừa**）
從零開始（參數全是亂數）	4 / 90	19
從 COCO 練好的模型開始	80 / 90	72

看上排。從零開始的模型像剛出生的嬰兒：參數全部是亂數，連邊都分不出來，只是把照片變成灰階。COCO 練好的原版模型像個大人：它看過 11.8 萬張照片，第一層就把樹、大樓、巴士、車道線的邊和輪廓都挑出來了。這就是「它已經會看」。再看下排：從零開始，90 個三角錐只框對 4 個；從 COCO 開始，框對 80 個。車和行人我們也試過，從零開始一樣學不會，53 個框對 0 個。

> **兩個名詞，先分清楚**
>
> 遷移學習（Học chuyển giao, transfer learning）：把別人已經學會的本事，借來用在新的事情上。
>
> 微調（Tinh chỉnh, fine-tuning）：借來之後，用自己的照片接著練的那一步。這次我們的做法就是微調：借 COCO 練好的模型，再用 30 張三角錐照片接著練。

就像學騎機車。從來沒騎過腳踏車的人，要先學平衡、學轉彎，學很久。會騎腳踏車的人，平衡已經會了，只要再學油門和煞車，很快就上手。

一輪一輪練：猜、對答案、改一點點

想像你在練投籃：投一球，看球偏左還是偏右，下一球手腕調一點點，再投，再調。機器學習也是這樣：猜、對答案、改一點點，重複很多次。一輪（epoch）就是 30 張訓練照片全部看一遍，像把一整本習題本從頭寫一遍。每一輪裡有三件事：

1. 猜：模型在每一張照片上猜框。
2. 對答案：跟標準答案比，算出錯了多少。這個「錯多少」叫損失（loss），可以想成考卷上被扣的分數。
3. 改一點點：把模型裡 260 萬個參數，每一個都往「錯少一點」的方向改一點點，就像投籃時身體 260 萬個小地方各調一點點。

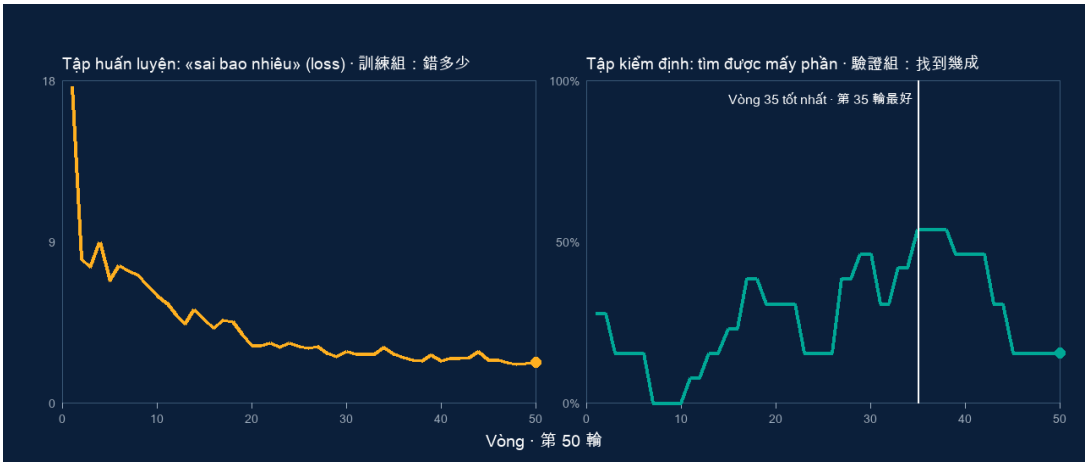
電腦一次看 4 張照片（叫一批），所以 30 張是 8 批，等於一輪。這次練 **50 輪**：30 張照片，每一張都看了 50 次，而且每一次都經過資料增強，樣子都不太一樣。



同一張波士頓測試照片（訓練時沒看過）。上：第 **1** 輪；下：第 **35** 輪。白色虛線 = 標準答案（**8** 個），綠 = 框對，紅 = 多框。右邊的數字是這一張和 **17** 張測試照片合計的結果。為了教學，這裡每一輪都拿測試照片考；真正做的時候，每一輪只能看驗證組。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

- **第 1 輪**：照片上到處都是紅框，連路面、樹都框。17 張測試照片，多框了 **3,030 個**。它剛開始學，只知道「橘色、尖尖的可能是」，就到處亂猜。
- **中間幾輪**：忽好忽壞。有幾輪幾乎什麼都不框，有幾輪又到處亂框。這是正常的：訓練不是每一輪都進步，整體往好的方向走就好。
- **第 19 輪以後**：慢慢穩定下來，紅框愈來愈少。**第 35 輪**：這一張 8 個三角錐全部框對，多框 4 個；17 張測試照片，90 個三角錐框對 80 個，多框 72 個。

練習題愈做愈好，模擬考先升、後降



訓練時軟體自動記下的兩條曲線。左：訓練組的損失（練習題被扣多少分），第 **1** 輪 **17.6**、第 **20** 輪 **3.2**、第 **50** 輪 **2.3**。右：驗證組（**8** 張、**13** 個三角錐）找到幾成（召回率）；白線 = 第 **35** 輪。

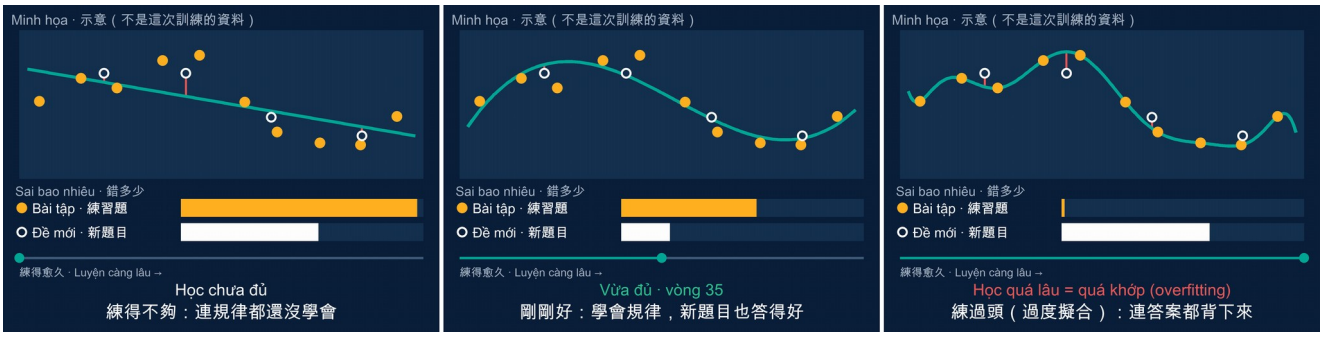
左邊，練習題的損失一路往下：練習題愈做愈熟。右邊，驗證組的成績就是模擬考：13 個三角錐，找到 7 個就是 54%。一開始兩、三成，中間有幾輪掉到 0，後來慢慢往上，**第 35 到 38 輪最高，54%**；可是再練下去反而往下掉，第 50 輪只剩 15%。軟體每一輪都算驗證組的綜合分數，自動把最高的那一輪存起來，就是第 35 輪。第 2 節用的模型，就是這一輪。

要提醒一點：驗證組只有 13 個三角錐，多找到或少找到一個，成績就差 8%，所以這條線跳得很厲害。驗證組太小是這次實驗的限制；真正做的時候，驗證組要大很多。

練過頭 = 過度擬合：背答案

為什麼練習題一直進步，模擬考反而退步？想像一個同學準備考試，只寫同一本考古題。寫第一遍，他學到觀念；寫到第十遍，他已經記得「第 3 題答案是 B」「第 7 題答案是 C」。考古題他每一題都對，可是期中考換了新題目，他反而不會，因為他記的是答案，不是觀念。

模型也會這樣。30 張照片看了 50 次以後，它開始記住這 30 張照片的細節：新加坡那段路的顏色、那幾個三角錐擺的位置。它在練習題上錯得愈來愈少，可是換到波士頓的照片，就認不太出來了。這叫過度擬合（overfitting）。



示意圖（不是這次訓練的資料）。黃點是練習題，白圈是沒看過的新題目，青綠色的線是模型的答法，下面兩條長條是「錯多少」。左：練不夠；中：剛剛好；右：練過頭。

- 練不夠（欠擬合，underfitting）：線只是一條直線，連大概的走向都沒學會，練習題和新題目都錯很多。
- 剛剛好：線抓到大概的走向，新題目錯得最少。我們留的就是這一類，第 35 輪。
- 練過頭（過度擬合）：線繞來繞去，硬是穿過每一個黃點，練習題的錯變成零；可是白圈離線更遠了，新題目又錯得更多。連練習題裡的小誤差，也一起背下來了。

所以我們留**第 35 輪**，不留最後一輪。拿測試照片驗證：90 個三角錐，第 35 輪框對 **80 個**，第 49 輪只框對 **66 個**。不過也要老實說，第 49 輪的多框也比較少，從 72 個降到 28 個：它變得比較保守，沒把握的就不框。這種「框得少、也漏得多」的取捨，第 2 節會再談。

防止練過頭的方法：照片多一點、資料增強多一點、用驗證組看什麼時候該停。

第 1 節要記住的三句話

一、一個東西一個框。整段影片分三組：訓練（練習題）、驗證（模擬考）、測試（期中考，最後只用一次）。

二、不從零開始（遷移學習）：借練好的模型，再用自己的照片微調。每一輪：猜、對答案、改一點點。

三、練過頭 = 背答案。留模擬考（驗證組）成績最好的那一輪，這次是第 35 輪。

三、推論與打分數（第 2 節）

推論：一張照片猜 19,320 次，篩到只剩 16 個框



波士頓的一張測試照片（**scene-0655 #34**，訓練時沒看過），模型是第 **35** 輪。右邊是五個步驟，照片上的框跟著一步一步變少。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

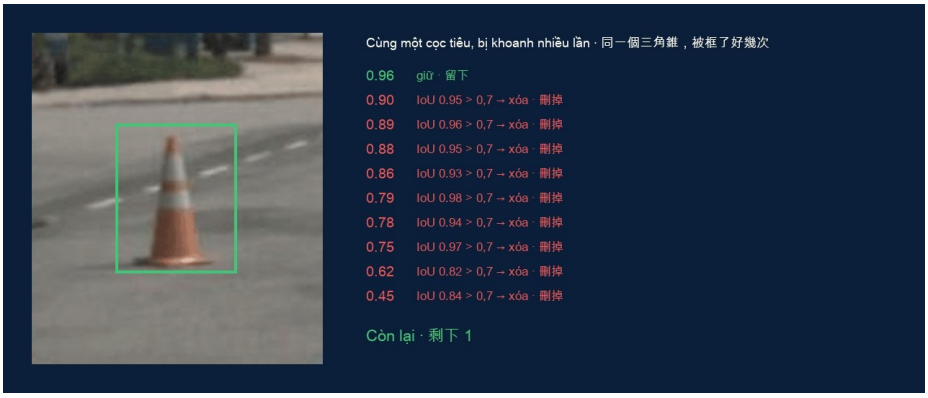
推論就是練好的模型真正拿來用：一張新照片進來，框出三角錐。可以把它想成：照片上站了一萬九千多位小觀察員，每個人只負責盯自己那一小塊，然後各自報告：「我這裡有沒有三角錐、框多大、我有幾成把握。」每一個猜測有一個框和一個分數，分數就是那幾成把握。

表 3　推論的五個步驟（scene-0655 #34）

步	名稱	做什麼	這張照片的數字
1	縮照片	縮到 1280 寬，跟訓練時一樣	1280
2	猜	照片上所有位置同時各猜一個框和一個分數	19,320 個猜測
3	看分數	大部分猜測的分數幾乎是 0：「這裡沒有三角錐」	分數 ≥ 0.01 的有 244 個
4	過門檻	分數低於門檻 0.25 的不算	剩 89 個
5	刪重複（NMS）	同一個三角錐被好幾個位置各框了一次，只留分數最高的	剩 16 個

要注意：真正的模型不是一塊一塊掃。整張照片只送進去一次，19,320 位觀察員同時報告，像全班同時交卷。動畫放慢，只是為了讓大家看到「每個位置都猜了一次」。這就是 YOLO 說的「只看一次」。

刪掉重複的框（NMS）



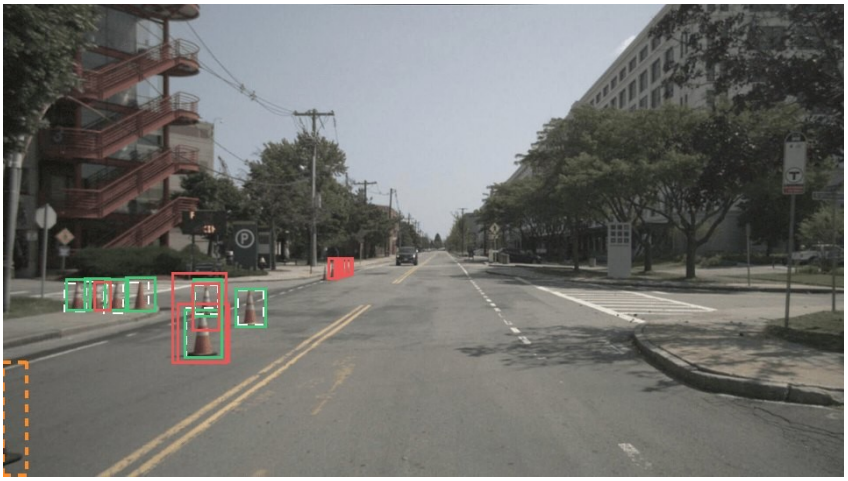
放大一個單獨的三角錐：分數 **0.25** 以上的框有 **10** 個。留下分數最高的 **0.96**，其他 **9** 個和它的 **IoU** 都在 **0.82** 到 **0.98** 之間，全部刪掉，剩 **1** 個。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

一個三角錐，好幾個格子、好幾層都覺得「這裡有三角錐」，每一個都猜了一個框。10 個框全部留下來，自駕車會以為那裡有 10 個三角錐。就像同一場火災有十個人打電話報案，消防隊只會當成一件事。做法有三步：

1. 找分數最高的，留下來（0.96）。
2. 其他的框，一個一個跟它比 IoU（重疊多少）。重疊超過 0.7，代表框的是同一個東西，刪掉。
3. 剩下沒被刪的，再找分數最高的，重複一樣的事，直到沒有框。

這個方法英文叫 **NMS**（non-maximum suppression），意思是「不是最大的，就壓下去」。IoU = 重疊的面積 ÷ 兩個框合起來的面積（第八週）。NMS 也會出錯：兩個三角錐靠得很近，框重疊超過 0.7，可能被當成同一個，刪掉一個；同一個三角錐的兩個框，大小差很多、重疊不到 0.7，就兩個都留下來，變成多框。

打分數：框對、多框、漏掉



同一張照片。白色虛線 = 標準答案 **8** 個；模型框了 **16** 個：框對 **7**（綠）、多框 **9**（紅）、漏掉 **1**（橘色虛線，在最左下角，三角錐被照片的邊切掉，只露出一小塊）。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

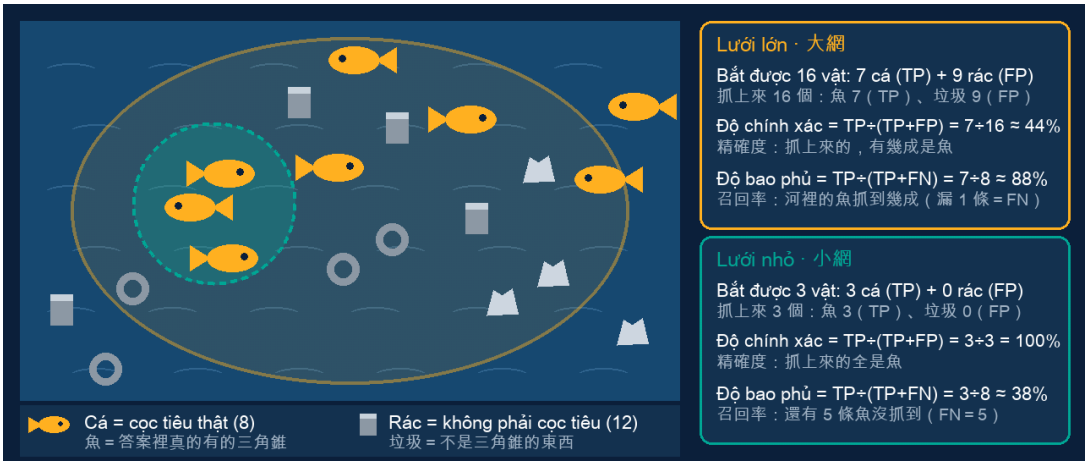
推論完，最後一步是打分數：拿模型的框，跟標準答案一個一個比。**IoU** 在 **0.5** 以上，才算框對（第八週的標註練習台用 0.3，比較寬鬆；正式比賽常用 0.5）。一個答案只能配一個框，多出來的都算多框。

表 **4**　三種結果（簡稱、英文名、意思）

簡稱	名字	意思	這張照片
TP	框對 true positive	模型框了，答案真的有（IoU ≥ 0.5）	7
FP	多框 false positive	模型框了，答案其實沒有	9
FN	漏掉 false negative	答案有，模型沒框到	1

這三個名字，用新冠快篩來記：出現兩條線叫陽性（**positive**，驗出有），只有一條線叫陰性（**negative**，沒有）。說對了叫 **true**，說錯了叫 **false**。多框（FP）：模型說有、其實沒有，像快篩兩條線，人其實沒染病，叫假陽性。漏掉（FN）：模型說沒有、其實有，像人真的染病，快篩卻只有一條線，叫假陰性。還有第四種 true negative（TN，說沒有、真的沒有），偵測用不到它，補充裡的混淆矩陣會講。

兩個分數：精確度與召回率



撒網捕魚。魚 = 答案裡的三角錐（**8** 條），垃圾 = 不是三角錐的東西（**12** 件）。大網抓到 **16** 個（**7** 魚 + **9** 垃圾），小網抓到 **3** 個（全是魚）。數字和上一張相同：框對 **7**、多框 **9**、漏掉 **1**。

把上一張的數字換成一條河：河裡有 8 條魚（答案裡的 8 個三角錐），還有 12 件垃圾（不是三角錐的東西）。

精確度 precision = TP ÷（TP + FP）　　**召回率 recall = TP ÷（TP + FN）**

表 5　大網與小網

網	抓上來	精確度	召回率
大網	16 個：7 魚 + 9 垃圾（漏 1 條）	7 ÷ 16 ≈ 44%	7 ÷ 8 ≈ 88%
小網	3 個：全是魚（漏 5 條）	3 ÷ 3 = 100%	3 ÷ 8 ≈ 38%

- 精確度問：抓上來的東西，有幾成是魚？也就是模型框出來的，有幾成是對的。大網只有 44%，抓上來的大部分是垃圾，框出來的不太可信。
- 召回率問：河裡的魚，抓到幾成？也就是該找的，找到幾成。大網 88%，幾乎沒有漏。

兩次擺在一起看：大網，召回率高、精確度低；小網，精確度高、召回率低。網撒得愈大，愈不會漏魚，也愈容易撈到垃圾；網撒得愈小，撈上來的愈乾淨，也愈容易漏魚。請注意兩個分數的分母不一樣：精確度的分母是「抓上來的全部」（模型框了幾個），召回率的分母是「河裡全部的魚」（答案有幾個）。網的大小，在模型裡就是分數門檻。

門檻：一個分數變高，另一個就變低

門檻可以想成錄取分數線。分數線訂得很低，比方說 20 分，幾乎每個人都錄取，好學生一個都不會漏掉，可是混進來不合格的人很多。分數線訂得很高，比方說 90 分，錄取的人都很優秀，可是很多不錯的學生被擋在門外。



左：同一張照片；右：**17** 張測試照片合計（**90** 個三角錐）。上：門檻 **0.10**，精確度 **39%**、召回率 **92%**；下：門檻 **0.90**，精確度 **82%**、召回率 **10%**。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

表 6　門檻表：**17** 張測試照片、**90** 個三角錐

門檻	框對	多框	漏掉	精確度	召回率
0.05	83	206	7	29%	92%
0.25	80	72	10	53%	89%
0.5	74	42	16	64%	82%
0.7	49	21	41	70%	54%
0.9	9	2	81	82%	10%

看多框那一欄：門檻 0.05 是 206 個，0.25 是 72 個，0.7 只剩 21 個，門檻愈高，多框愈少。看漏掉那一欄：0.05 只漏 7 個，0.25 漏 10 個，0.7 漏 41 個，0.9 漏 81 個，門檻愈高，漏掉愈多。

精確度和召回率就是從這幾欄算出來的。例如門檻 0.25：框對 80、多框 72，一共框了 152 個，80 ÷ 152，精確度 53%；90 個三角錐框對 80 個，召回率 89%。

請看 0.25 和 0.5 兩列：門檻從 0.25 調到 0.5，多框少了 30 個，漏掉只多了 6 個，好像很划算。可是從 0.5 調到 0.7，多框少 21 個，漏掉卻多了 25 個。門檻沒有「最好」的值，只有「看你要什麼」，要在這種取捨裡選一個。

自駕車選哪邊：寧可多框，但多框太多也會出事



示意模擬（不是實測）：時速 **50 km**；後車跟車約 **14 m**（**1** 秒）、反應 **1** 秒、最大減速 **5 m/s²**。條件改了，結果會變。

這一段不是說召回率永遠比精確度重要，是說兩種錯的代價不一樣，所以偵測這一關的門檻，通常往「寧可多框」放。看動畫，三列的車都是時速 50 公里：

- 漏掉：前面 40 公尺有一個行人，模型沒框到，車子不知道有人，照樣開，2.7 秒以後撞上去。這是最嚴重的結果。
- 多框一次：前面飄著一個塑膠袋，被誤框成障礙物，車子煞一下再加速，晚了大約 1.6 秒。後車跟 14 公尺，有反應時間，剛好沒撞上。代價是一點時間和乘客的不舒服。
- 多框太多：同樣的誤框，每 3 秒來一次。第一次後車還跟得上；第二次後車還沒從上一次恢復，煞車又來了，追撞。前面其實什麼都沒有，是自己亂煞。

所以要壓低的是兩種錯一起。實務上有三個辦法：（一）偵測先寧可多框，但不要馬上緊急煞車；（二）後面把假的濾掉：一個框要連續好幾張都出現才算數，相機、雷達、光達互相確認，這是第 3 節追蹤的內容；（三）不同的動作用不同的把握：把握不高，只是放油門、輕輕減速；把握很高，才全力緊急煞車。就像火災警報器，寧可敏感一點，可是不能每次煎魚都叫消防隊來。

答案也會出錯：標準答案是人標的



門檻 0.25、17 張測試照片：72 個多框裡，有 52 個框在官方沒標任何三角錐的位置，一個一個看過，框裡都是三角錐；這裡挑分數最高的 8 個。橘色虛線 = 答案有標的三角錐，紅框 = 模型的框（分數 0.85–0.95）。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

精確度 53%，聽起來很差：框出來的，將近一半是錯的。可是把這些「錯的」一個一個裁出來看：**52 個**框的位置，答案沒標，框裡卻真的是三角錐，模型的分數也很高。不是模型錯，是答案漏標了。就像老師改考卷，標準答案寫錯了一題：你寫對了，卻被扣分。這 52 個如果算對，精確度從 53% 變成 87%（(80 + 52) ÷ 152 ≈ 87%）。

標註是人標的，訓練用的照片也是同一套標法，一樣可能漏標。漏標的三角錐放進訓練，模型會學到「那裡沒有三角錐」。所以標註資料是模型的課本，也是考卷的標準答案：答案有錯，模型學到的和拿到的分數都會受影響。

72 個多框其實是這樣組成的：52 個是答案漏標；另外 20 個框在三角錐上，但和答案的框重疊不到一半，所以不算框對（框的大小跟答案不一樣，或同一個三角錐被框了兩次）。漏掉的 10 個裡，有 4 個在照片最邊邊，三角錐只露出一小塊。

資料引擎：訓練不是做一次就結束

這個循環，就像你考完試以後做的事：把錯的題目抄進錯題本，看出自己哪裡弱，針對那裡補強、再練，然後再考一次。

1. 找出錯的：發現答案本身漏標了 52 個；漏掉的 10 個裡，有 4 個在照片邊邊。

2. 補資料、修答案：今天的模型只看過新加坡一段路、白天、晴天。要補它沒看過的：其他城市、晚上、下雨、被車擋住一半的三角錐。漏標的要請人補標，不然下一次訓練，模型會學到「那裡沒有三角錐」。
3. 再訓練：一樣從練好的模型開始，一樣用驗證組挑最好的一輪。
4. 再考一次：用模型沒看過的照片考，然後又會找到新的錯，再回到第一步。

自駕車公司把這個循環跑幾百次，叫資料引擎（data engine）。感知做得好不好，很大一部分不是靠模型的設計，而是靠這個循環轉了多少次。「資料引擎」是業界的說法（Tesla 的 AI 團隊談自動駕駛時用它，Meta 的 Segment Anything 論文也把自己的標註流程叫 data engine），不是學術上的標準術語。學術上相近的概念有：主動學習（active learning，讓模型挑出最沒把握的資料請人標）、難例挖掘（hard example mining，專門找模型最容易錯的例子來訓練）、以資料為中心的 AI（data-centric AI，改資料，不只改模型）。看論文搜這幾個關鍵字，看公司的簡報就會看到 data engine。

補資料也要補對：重教已經會的，反而忘了

三角錐是原版模型不會的。如果拿它原本就會的東西，車和行人，用少量照片再教一次呢？我們也試了。訓練照片 30 張，從 nuScenes 七段不同的影片挑出來，有 107 台車、30 個行人；測試照片 10 張：波士頓白天的斑馬線 5 張，加上新加坡晚上剛下過雨的路口 5 張。

表 7　各種做法框對幾個（10 張測試照片）

做法	車（19）	行人（30）	兩輪（4）
原版 COCO 模型，不再教	13	13	1
30 張，從零開始	0	0	0
30 張，從 COCO 微調	11	0	0
282 張，從 COCO 微調	12	7	1

行人從 13 個掉到 0 個。它原本會認人，被我們用 30 張照片「教」了以後，反而不會了：這 30 張裡只有 30 個行人，太少，為了把這 30 張練好，模型把原本從十一萬張照片學來的東西改掉了。這就像有一個月，你只拼命背 30 句考試會考的句子，其他日常的中文反而講得不順。用 282 張，行人回到 7 個，還是比原版的 13 個差。

這告訴我們兩件事。第一，訓練不是一定會變好，資料太少，可能把原本會的教壞。第二，自己訓練最值得做的，是像三角錐這樣，教它原本不會的東西。越南路上的三輪貨車（xe ba gác），COCO 和 nuScenes 都沒有這一類，就是值得自己收資料、自己訓練的地方。

推論要多快

推論不只要準，還要快。在老師的 MacBook 上，一張照片大約 **0.01** 秒（精確一點是 0.0115 秒）。可是車上不是一張照片：nuScenes 的車有 **6** 顆相機，每一顆一秒拍 **12** 張，一秒就是 6 × 12 = 72 張。72 × 0.0115 秒 ≈ **0.83** 秒：一台 MacBook 全力跑，一秒鐘剛好跑得完這一秒拍的照片。

聽起來夠了，但這還只是三角錐一樣東西。車上還要找車、找人、找車道線、看紅綠燈，每一樣都要算；而且車子不能「剛好跟得上」，要留時間給後面的預測和規劃。所以真的自駕車上，有專門為這件事設計的晶片（第十五週會講車載運算平台）。

第 2 節要記住的三句話

一、推論：19,320 個猜測 → 門檻 → 刪重複 → 剩幾個框。跟答案比：框對、多框、漏掉。

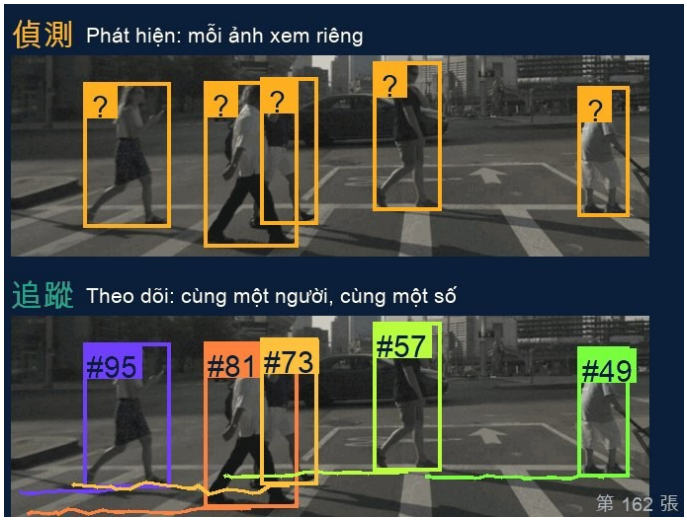
二、精確度 = 框對 ÷ 框了幾個；召回率 = 框對 ÷ 真的有幾個。門檻愈高，多框愈少、漏掉愈多。

三、分數只跟答案一樣準。一個一個看錯在哪、補對的資料、再訓練，這就是資料引擎。

四、追蹤：從一張照片到一段影片（第 3 節）

前兩節，模型都是一張照片一張照片地看。這一節的例子全部是同一段真實影片：nuScenes 的 scene-0553，波士頓 Summer St. 的一個路口。車子停在斑馬線前面，整段 20 秒都沒有動，很多行人從左邊走過斑馬線到右邊。前方相機一共拍了 237 張，大約每秒 12 張。畫面上的框是第 1 節那個原版模型（COCO，認得人）框的，框旁邊的號碼是一個簡單的配對程式給的。

偵測和追蹤：一張一張分開看 vs 每個東西一個號碼

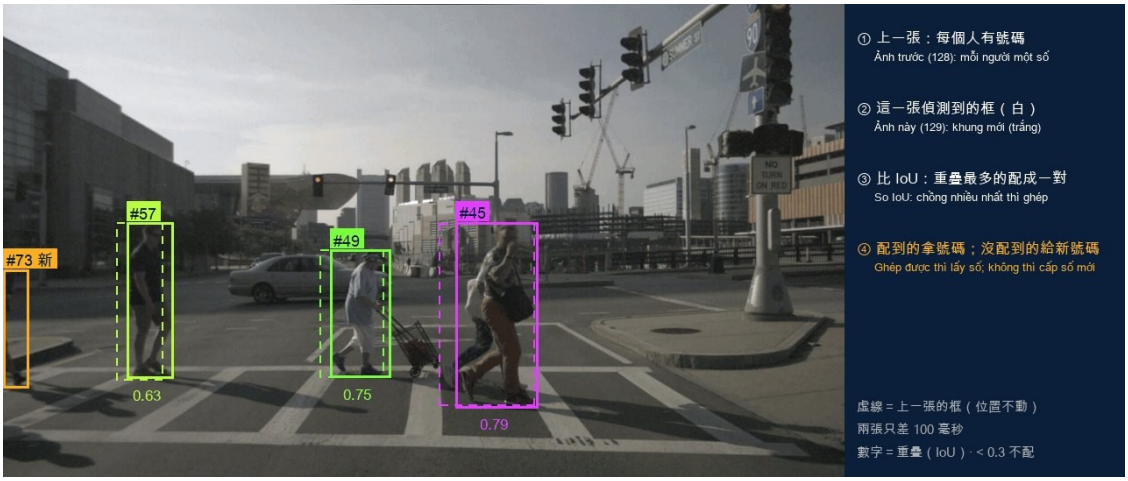


上：偵測，每一張各自把人框出來，框上只有問號；下：追蹤，同一個人一直用同一個號碼，腳底的線是走過的路（**scene-0553 第 148–162 張**）。　資料：**nuScenes（Motional）**，**CC BY-NC-SA 4.0**

週末晚上你跟朋友逛夜市，人很多。你每看一眼，都看到很多人：「這裡有人、那裡有人」，這是偵測。可是你要跟著朋友走，就要知道這麼多人裡面，哪一個是你的朋友。你記得他穿紅色的衣服、剛才在你左前方，所以下一眼，你往左前方找穿紅衣服的人。這就是追蹤。

車子也一樣。偵測只回答「這一張哪裡有人」；追蹤再回答「這個人，是不是上一張那一個」，是的話，就給他同一個號碼。為什麼一定要知道？一個行人站在路邊，只看一張照片，你不知道他是站著等，還是正要走到馬路上。要看好幾張，知道同一個人從哪裡走到哪裡，才知道他往哪個方向走、走多快。車子要不要減速，就靠這個判斷。

配對：拿上一張的框，跟這一張的框比重疊



scene-0553 第 128 張 → 第 129 張（相差 0.1 秒）。虛線 = 上一張的框（位置不動），數字 = 上一張的框和這一張的框重疊多少（IoU），不到 0.3 不配。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

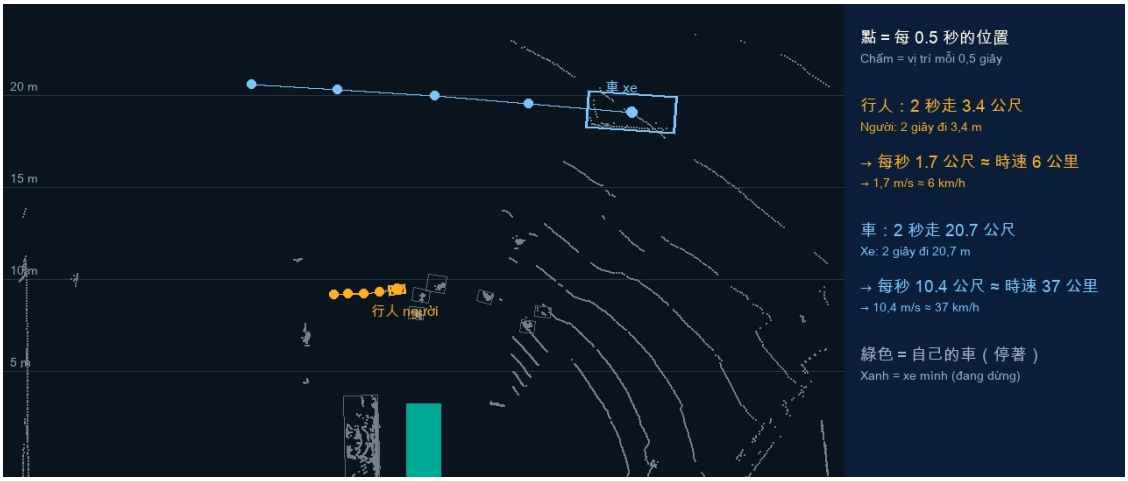
這兩張照片只差 0.1 秒。配對的做法，像小時候玩的連連看：上一張的每個人是一個點，這一張的每個框是另一個點，把重疊最多的連起來，連到的，就拿到那個人的號碼。

1. 上一張：每個人都已經有號碼：57、49、45。
2. 這一張：模型重新把所有的人框出來（白框），這時白框還沒有號碼。上一張的框用虛線留在原來的位置，顏色跟號碼一樣。0.1 秒很短，人只走了一小段，所以虛線和白框幾乎疊在一起。
3. 比 **IoU**，重疊最多的配成一對：57 號的虛線框，跟它重疊最多的白框，IoU 是 0.63；45 號是 0.79；49 號是 0.75。重疊不到 0.3，就不配。
4. 號碼傳下去：白框跟誰的虛線框重疊最多，就拿誰的號碼。左邊邊緣剛走進來一個人，他沒有跟任何虛線框重疊，就給他一個新號碼 **73**。

每一張照片都重複這四步，一秒鐘做 12 次。

> 補充：還有更準的做法
>
> 真正的系統會先猜再比：看前兩張，這個人的框往哪個方向移了多遠，就把框照樣挪一步，再去比 IoU。57 號直接比是 0.63，先挪再比是 0.94。用四段影片（每秒 12 張）試過，不先猜，產生的號碼多 2% 到 10%，也就是比較常斷掉、重新編號。差別不大，所以這一節只講直接比。第十二週的卡曼濾波，就是更準的猜法。

有了號碼，才算得出速度



同一個路口的鳥瞰圖（**scene-0553** 第 **15** 秒）。位置 = **nuScenes** 官方標註（每 **0.5** 秒一次），灰點 = 光達，綠色長方形 = 自己的車（停著）。橘色：行人，**2** 秒走 **3.4** 公尺；藍色：車，**2** 秒走 **20.7** 公尺。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

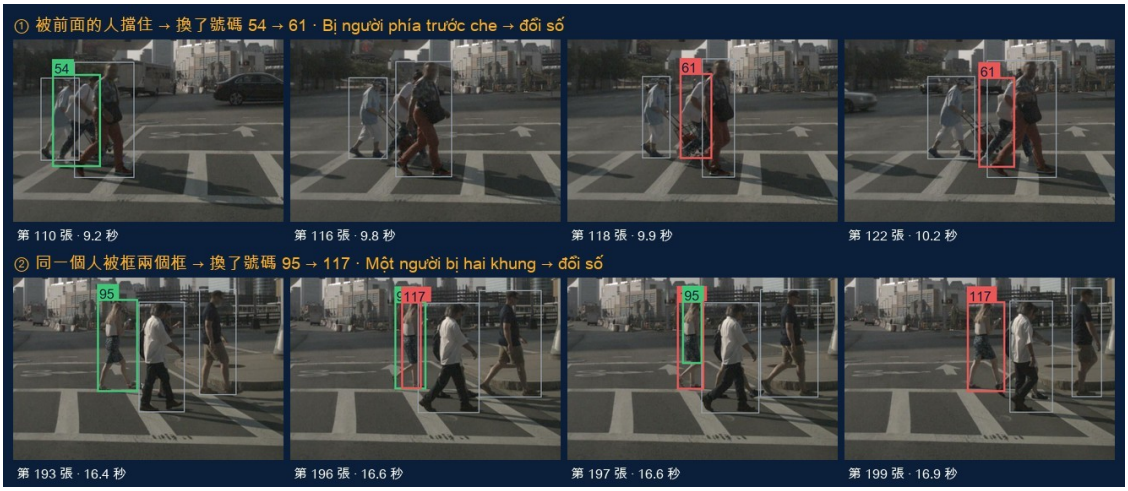
高速公路的區間測速：兩個測速點相隔 5 公里，同一台車 3 分鐘開完，時速就是 100 公里。要算得出來，得先確定前後兩個點拍到的是同一台車，靠的是車牌。追蹤的號碼，就是行人和車的車牌。

速度 = 同一個號碼走的距離 ÷ 時間

- 橘色是一個行人，正在走斑馬線，2 秒走了 3.4 公尺。3.4 ÷ 2 = 每秒 1.7 公尺，大約時速 **6** 公里，就是一般人走路的速度。
- 藍色是一台車，在前面的路口橫著開過去，同樣 2 秒走了 20.7 公尺，每秒 10.4 公尺，大約時速 **37** 公里。所以車的點拉得很開，行人的點很密。

為什麼看 2 秒，不只看 0.5 秒？因為每一次的位置都有一點點誤差。這個行人如果只看最後 0.5 秒，算出來是每秒 1.9 公尺，偏快了。時間拉長一點，誤差就被平均掉。還有一點：速度要用地上的公尺來算，不能用照片上的像素。照片上近的東西大、遠的東西小，一個像素在近處和遠處代表的距離不一樣。這也是為什麼要有光達、要畫鳥瞰圖。

追蹤也會錯：被擋住、被框兩次，號碼就換了

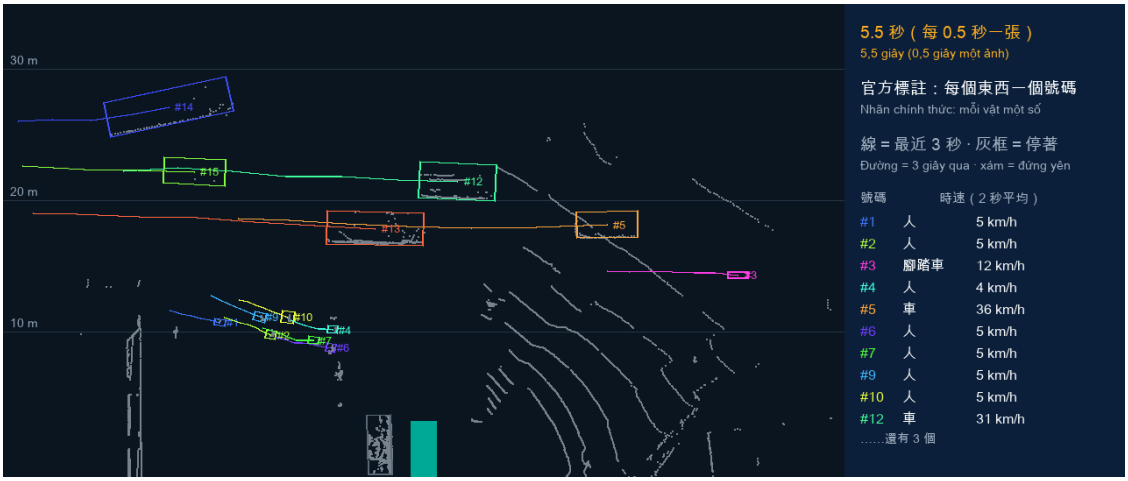


兩排都是同一段影片裡真的發生的錯誤。上：**54** 號被前面的人擋住，再出現時拿到新號碼 **61**。下：**95** 號的女士身上出現兩個框，多的那個拿到 **117**，**95** 從此消失。綠 = 原本的號碼，紅 = 換成的新號碼。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

- 被擋住：54 號是走在拉推車那位女士後面的一位男士。他被前面的人整個擋住，模型框不到他；過了大約 0.2 秒他又露出來，模型也框到了。可是程式拿他最後出現的位置，跟他現在的位置比，只重疊 0.10，不到 0.3，就當他是一個新的人，給了新號碼 61。
- 同一個人被框兩次：95 號是一位女士。模型在她身上框了兩個框，一個大、一個小，第 2 節講過的刪重複那一步沒有刪掉。多出來的那個框拿到新號碼 117；下一張 117 配到她整個人的框，95 只配到上半身，再下一張 95 就沒了，從此她變成 117 號。

號碼換了，有什麼關係？就像在夜市，你盯著穿紅衣服的朋友，一個大叔擋住他兩秒，再出現的時候你愣一下：「咦，這是誰？」你把他當成陌生人了。系統也一樣，會以為 54 號消失了，又出現一個剛走進來的 61 號。54 號剛才走過的路、他的速度，全部不見了，要從頭重新算。這在機車很多的地方最常發生：一台機車被另一台擋住，一下子又鑽出來，就可能變成另一個號碼。

在鳥瞰圖上追蹤：距離是真的公尺，人和車不會互相擋

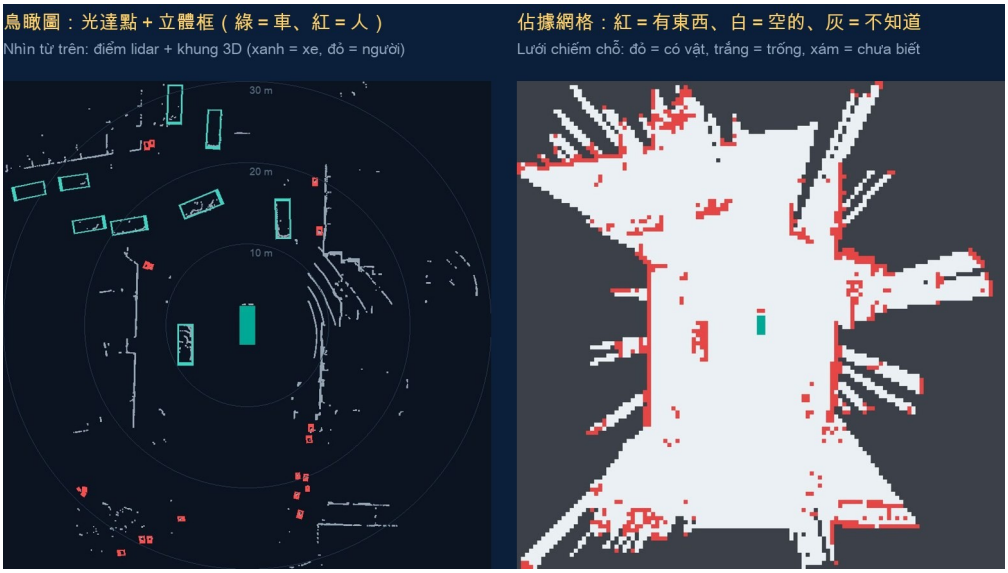


scene-0553 的官方標註，每 **0.5** 秒一張、共 **20** 秒；這是第 **5.5** 秒。線 = 最近 **3** 秒走過的路，時速 = 最近 **2** 秒的平均。　資料：**nuScenes（Motional）**，**CC BY-NC-SA 4.0**

第八週講過鳥瞰圖：從上往下看，像籃球教練的戰術板，每個球員是一個小圓圈。它有兩個好處。第一，不會互相擋：照片上前面的人會擋住後面的人，從上面看，每個人各佔一塊地。第二，距離不會變形：像地圖的比例尺，一公尺就是一公尺。這兩個好處，剛好解決前面兩個問題：被擋住，以及像素不能直接換成公尺。所以真正的自駕車，多半把相機、光達的結果放到鳥瞰圖上，再在鳥瞰圖上追蹤。

這張動畫用的是 nuScenes 的官方標註：請人在每一張關鍵畫面上把每個人、每台車都標好，而且同一個東西整段都用同一個號碼。這就是追蹤的標準答案；研究的人拿自己的追蹤程式跟它比，看號碼換了幾次、漏了幾個。這 20 秒裡，前面這一塊一共出現 18 個號碼：走斑馬線的行人，時速大約 4 到 6 公里；路口開過去的車，大約二十到三十幾公里；灰色的框是停著的車。

追蹤的限制：認不出是什麼的東西，靠佔據網格



第八週第 1 節的圖（nuScenes scene-0103 #14 光達）。左：鳥瞰圖 + 立體框；右：佔據網格，每格 0.5 公尺。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

追蹤是接在偵測後面的。偵測框得到，追蹤才有東西可以配對；偵測框不到，追蹤也沒辦法。還記得第 1 節的原版模型嗎？它沒學過三角錐，14 個三角錐一個都沒框到，那當然也追不到。路上掉了一箱貨、一個沒看過的路障，也一樣。

所以要用第八週學過的佔據網格（occupancy grid）。把地面切成一格一格，每格半公尺，每一格只問一件事：這裡有沒有東西。紅色是有東西，白色是空的，灰色是看不到、不知道。它不用知道那是什麼，就像家裡的掃地機器人：不知道撞到的是桌腳還是椅子，只知道「這裡有東西，繞過去」。怎麼用，三步：

1. 光達每掃一圈，就更新這張網格：有光達點打到的格子標「有東西」；光線穿過去的標「空的」；被擋住的標「不知道」。
2. 把規劃的路線一格一格對照：路線上只要有一格是「有東西」，這條路就不能走，要換一條。
3. 看最近的「有東西」還有多遠，跟第四週學的煞停距離比：不夠停，就要減速或停下來；碰到「不知道」的格子，也要慢一點。

用三角錐來說：偵測框不到，追蹤也追不到；可是光達還是在那個位置打到點，那幾格就標成「有東西」，車子照樣繞開。網格也有限制：它只知道這裡有東西，不知道那是什麼，塑膠袋可以壓過去、小孩一定要停，網格分不出來；它也看不出東西有沒有在動。所以真正的自駕車兩種都用：認得出來

的人和車，用偵測加追蹤，知道它是什麼、往哪走、多快；認不出來的東西，至少佔據網格知道那裡不能開過去。

追蹤的結果交給誰：預測，然後規劃

步	名稱	回答什麼問題	哪一週
1	偵測	這一張哪裡有人、有車	第八週、這週
2	追蹤	每個東西一個號碼：在哪、往哪走、多快	這週
3	預測	接下來 3 秒，他會走到哪裡？	第十三週
4	規劃	車子要停、要讓，還是可以走？	第十三週

知道一個行人正以每秒 1.7 公尺的速度走向馬路，才能預測他接下來 3 秒會走到哪裡；沒有追蹤，就沒有這些數字，預測沒辦法做。第十二週講定位的時候，會講到卡曼濾波：先猜它在哪，再把猜的和看到的合起來，算得更準。像開車進隧道，GPS 沒有訊號，導航會照剛才的速度猜你在哪；出了隧道收到訊號，再把猜的位置修正過來。車子估計自己在哪裡用它，估計別人在哪裡也用它。

第 3 節要記住的三句話

一、偵測一張一張看；追蹤替每個東西一個號碼：上一張的框跟這一張的框比重疊，重疊最多的配成一對。

二、有了號碼才算得出速度：同一個號碼走多遠 ÷ 時間，要用鳥瞰圖上的公尺。

三、被擋住、被框兩次，號碼會換。認不出的東西，靠佔據網格。

五、台灣與越南：訓練資料要有本地的

「模型只會它看過的」，這句話是這一週的共同結論。這次的模型只看過新加坡白天的 30 張照片；拿到河內或台灣的路上，會怎麼樣？

訓練：會錯在哪，要補什麼

- 晚上、下雨、被機車擋住：它從來沒看過晚上的三角錐。晚上顏色不一樣、反光條會亮、車燈會照出眩光，很可能漏掉。河內和台灣的路上機車很多，三角錐常常只露出一半，今天的訓練照片裡卻大多是完整的。要補晚上、下雨、被擋住的照片，標註時框看得到的部分。
- 其他橘色的東西：只用 5 張照片練的模型（補充裡有那組實驗），看到橘色就框，多框了 587 個。路上橘色的東西很多：施工圍籬、反光背心、橘色的機車、招牌，30 張練出來的模型也可能被騙。要讓訓練照片裡也有這些東西，而且沒有框，讓模型知道「橘色不一定是三角錐」。
- 教它認三輪貨車（**xe ba gác**）：越南路上很多，國外的資料集沒有這一類。它的樣子變化比三角錐大得多（載的東西不一樣，前、側、後看也不一樣），所以至少要幾百張，白天、晚上、下雨、不同的城市都要拍。分組時整段影片放同一組，測試組最好用另一個城市或另一天的影片，像今天練習題在新加坡、考試在波士頓。訓練時從練好的模型開始，不要從零開始。

追蹤：河內的機車潮

波士頓的行人走得慢、彼此離得也遠。換成河內的機車潮，追蹤會在哪裡出錯？

- 靠得很近、互相擋住：就是「追蹤也會錯」那一節上面那一排的情況，而且會一直發生，像下課的走廊人擠人，盯著同一個同學很容易跟丟。一台機車被旁邊的擋住一下，再出現時位置對不上，就給了新號碼。兩台並排、框疊在一起，可能被當成同一台，或者兩台的號碼互換。
- 突然鑽出來、突然轉彎：這一節的做法，靠這一張的框跟上一張的框重疊。機車常常突然加速、切到旁邊的車道，框一下子移很遠，重疊不到 0.3，就配不上，或者配錯人。所以真正的系統要先猜他會去哪再比，還要搭配光達、雷達量到的距離和速度。
- 號碼換掉的後果：系統忘了這台機車剛才是怎麼騎的，速度要從頭算；剛出現的新號碼還沒有速度，預測就猜不準。結果可能是兩種：車子以為「突然冒出一台新的機車」而急煞；或者沒算出它正在切過來，煞車慢了一步。機車多的地方，追蹤做得好不好，直接影響安全。

六、課後作業：精確度、召回率與門檻

作業在網頁上做，一個人做，手機、電腦都可以，大約 30 到 40 分鐘：

https://ch-neural.github.io/MITUST-Principles-of-Self-Driving-Cars/week10/hw/

1. 開網頁：填姓名、學號。題目的數字每個人都不一樣（依學號算出來），所以不要抄別人的答案。
2. 做題目：計算題 3 題、網頁操作 4 題（第 7 題選做）。每一題按「檢查」，對了會打勾，不對可以改，可以檢查很多次；每題都有提示。
3. 產生作答單，傳 **LINE**：全部做完，按「產生作答單」，網頁會把你的答案、對不對、檢查了幾次整理成一頁。把這一頁截圖傳到 LINE 群組；有打叉的題目，作答單會印出應該是什麼，請對照訂正。

表 **8**　作業的七題

題	名稱	做什麼
1	夜間行人	兩個模型各算精確度和召回率，選偵測這一關該用哪一個
2	為什麼偵測不用正確率	一律回答「沒有行人」的模型：算正確率和召回率
3	追蹤算速度	同一個號碼 2 秒走的距離，換算成時速
4	門檻拉桿	第 2 節同一個模型、17 張測試照片：拖動門檻、抄數字、算兩個分數、找出「召回率至少 85%」的最高門檻
5	IoU 小遊戲	拖動、縮放橘框，讓 IoU 落在指定範圍，再算兩個情況的 IoU
6	追蹤配對	看一張 IoU 表，決定每個新框拿到哪個號碼
7	（選做）自動緊急煞車	1000 個行人候選的模擬分數，白天、夜晚：找「漏掉 0」的最高門檻與誤煞車次數

網頁不會把你的答案傳到任何地方，答案只存在你自己的手機或電腦裡。這份作業算在「實作與上機」成績裡，不設繳交期限。第 5 題的拖動在手機上如果拖不動，請改用電腦。

七、練習題與參考答案

每題只有一個答案。先自己做，再看後面的參考答案。

1. 下面哪一組照片，是這次訓練中唯一能用來改模型參數的？

   A. 訓練組　　B. 驗證組　　C. 測試組　　D. 三組都可以

2. 驗證組（模擬考）的成績，這次拿來做什麼？

   A. 直接改 260 萬個參數　　B. 讓人決定留哪一輪　　C. 加進練習題一起練　　D. 當作最後的期中考成績

3. 為什麼同一段影片的照片要放在同一組？

   A. 檔案比較好管理　　B. 相鄰兩張只差 0.5 秒、幾乎一樣，分開放會洩題，成績比真正的程度好看　　C. 這樣訓練比較快　　D. 這樣照片才不會重複

4. 只用 30 張照片，從零開始只框對 4 個，借 COCO 練好的模型開始卻框對 80 個。這種「借一個已經練好的模型」的做法叫什麼？

   A. 資料增強　　B. 刪重複　　C. 遷移學習　　D. 過度擬合

5. 借來模型之後，用自己的三角錐照片接著練的那一步，叫什麼？

   A. 推論　　B. 標註　　C. 微調　　D. 追蹤

6. 訓練組的損失一直往下，驗證組的成績卻在第 35 輪之後下降。最可能是怎麼回事？我們該留哪一輪？

   A. 門檻太高；留第 50 輪　　B. 過度擬合，模型在背練習題；留第 35 輪　　C. 驗證組太大；留第 1 輪　　D. 照片太亮；留第 50 輪

7. 一張照片，模型猜了 19,320 次，最後只剩 16 個框。中間做了哪兩件事？

   A. 縮照片、標註　　B. 過分數門檻、刪掉重複的框　　C. 資料增強、算損失　　D. 配對、算速度

8. 答案有 8 個三角錐，模型框了 16 個，其中 7 個和答案的 IoU 在 0.5 以上。框對、多框、漏掉各是幾個？

   A. 7、9、1　　B. 7、1、9　　C. 16、8、1　　D. 9、7、1

9. TP = 60，FP = 20，FN = 40。精確度和召回率是多少？

   A. 精確度 75%、召回率 60%　　B. 精確度 60%、召回率 75%　　C. 精確度 60%、召回率 40%　　D. 精確度 75%、召回率 40%

10. 把門檻從 0.25 拉到 0.7（17 張測試照片），多框從 72 降到 21，漏掉從 10 升到 41。下面哪一句對？

A. 門檻愈高，多框愈少、漏掉愈多　B. 門檻愈高，多框愈多、漏掉愈少　C. 門檻愈高，兩種錯都變少　D. 門檻和多框、漏掉沒有關係

11. 自駕車偵測這一關，門檻通常怎麼放？

A. 放很高，因為多框會害後車追撞　B. 偏低（寧可多框），因為漏掉行人最嚴重，多框的再靠追蹤和多種感測器濾掉　C. 一定要放 0.9　D. 門檻對安全沒有影響

12. 標準答案漏標了 52 個三角錐，但模型框到了。這 52 個會被算成什麼？

A. 多框，精確度被拉低（模型被冤枉）　B. 漏掉　C. 框對　D. 不影響成績

13. 追蹤配對時，這一張的框跟上一張某個框的 IoU 最大、又不小於 0.3，會怎麼處理？沒配到的框呢？

A. 拿那個號碼；沒配到的給新號碼　B. 拿新號碼；沒配到的刪掉　C. 兩種都拿 0 號　D. 拿那個號碼；沒配到的沿用最近的號碼

14. 行人 2 秒走了 3.4 公尺，大約時速多少？

A. 1.7 公里　B. 3.4 公里　C. 6 公里　D. 12 公里

15. 一個人被前面的人擋住，再出現時，和他最後出現的位置只重疊 0.10。照這一節的做法（不到 0.3 不配），程式會怎麼處理？

A. 當成新的人，給新號碼　B. 沿用舊號碼　C. 把他刪掉　D. 當成障礙物

16. 路上有一個從來沒看過的路障，偵測框不到它。哪個方法至少能讓車子知道那裡不能開過去？

A. 追蹤　B. 佔據網格　C. 刪重複（NMS）　D. 資料增強

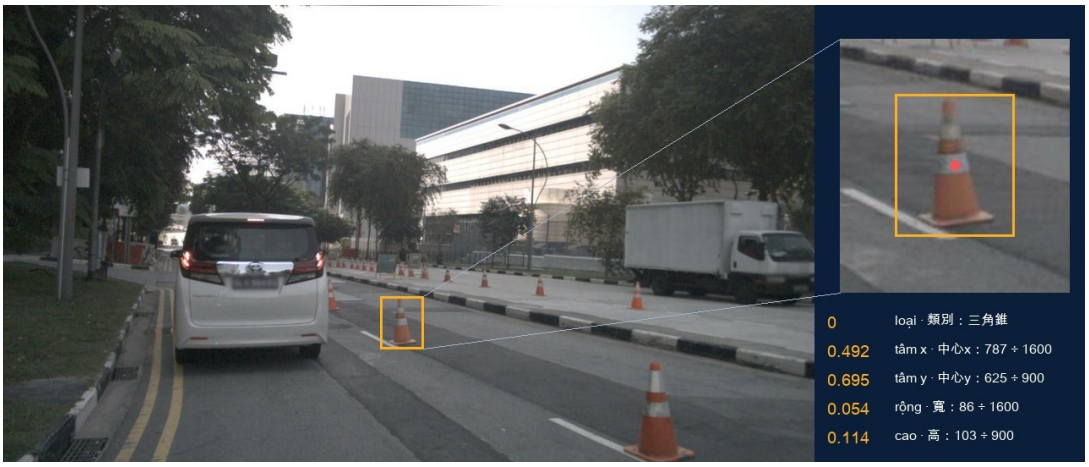
參考答案

題	答案	一句話說明
1	A	只有練習題（訓練組）能改參數；驗證組、測試組都不能。
2	B	成績是給人看的，人決定留哪一輪；模型不會因為考了模擬考而改變。
3	B	相鄰兩張幾乎一樣，一張在練習題、一張在考卷，就是資料洩漏。
4	C	把別人已經學會的本事借來用，叫遷移學習。
5	C	借來之後用自己的照片接著練的那一步，叫微調。

題	答案	一句話說明
6	B	練習題愈來愈好、模擬考變差 = 過度擬合；留驗證組最好的第 35 輪。
7	B	19,320 個猜測 → 分數門檻（剩 89 個）→ 刪重複（剩 16 個）。
8	A	框對 TP = 7；多框 FP = 16 − 7 = 9；漏掉 FN = 8 − 7 = 1。
9	A	精確度 = 60 ÷ (60 + 20) = 75%；召回率 = 60 ÷ (60 + 40) = 60%。
10	A	門檻拉高：多框 72 → 21 變少，漏掉 10 → 41 變多。
11	B	漏掉行人最嚴重，所以偏低；多框太多也會出事，要靠追蹤、感測器融合濾掉，不同動作用不同的把握。
12	A	答案漏標，模型框對了卻被算成多框，被冤枉；算對的話精確度從 53% 變成 87%。
13	A	重疊最多（不小於 0.3）的拿號碼；沒配到的是新來的，給新號碼。
14	C	3.4 ÷ 2 = 1.7 m/s；1.7 × 3.6 ≈ 6 km/h。
15	A	重疊 0.10 < 0.3，不配，當成新的人，給新號碼（61 號）；前面走過的路就不見了。
16	B	佔據網格不問是什麼，只問這一格有沒有東西；路線上有「有東西」的格子，就不能經過。

八、補充：訓練與推論的細節（上課不講，備查）

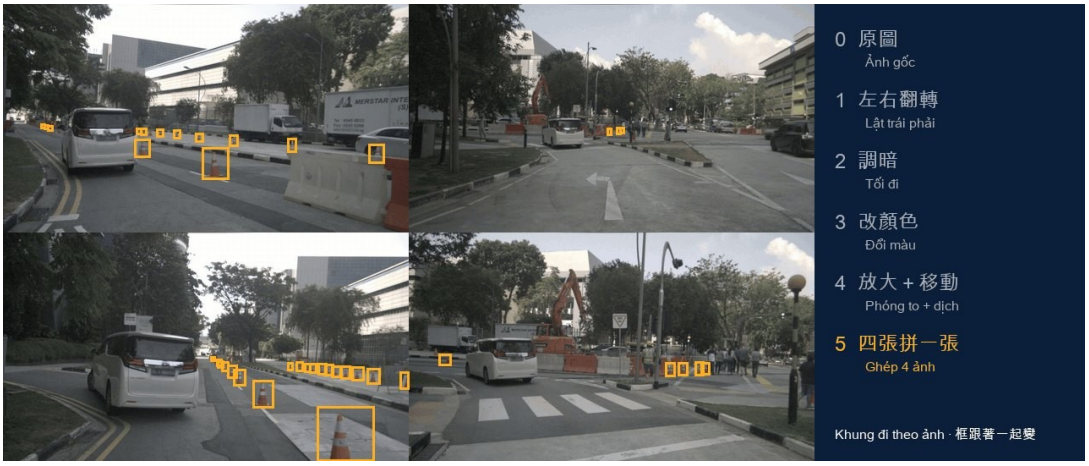
一個框存成五個數字



scene-0061 #23 中間那個三角錐。照片原本 **1600 × 900** 像素，右上是放大，紅點是框的中心。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

YOLO 格式每一行是：類別、中心 x、中心 y、寬、高。類別：這次只教一種，三角錐是 0。中心 x：紅點在左邊數過來第 787 個像素，787 ÷ 1600 = 0.492。中心 y：紅點在第 625 個像素附近，625 ÷ 900 ≈ 0.69，檔案裡存的是 0.695（這裡的像素位置只取整數）。寬：86 個像素，86 ÷ 1600 = 0.054。高：103 個像素，103 ÷ 900 = 0.114。為什麼都要除以照片的寬和高？因為訓練時照片會被縮小、放大；記成比例（0 到 1 之間），照片放多大，數字都還是對的。

資料增強：一張照片變出很多張



資料增強的示意（最後一格：四張拼一張）。照片怎麼變，標註的框就跟著一起變。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

只有 30 張照片太少，所以訓練時每一輪都隨機把照片變一次：左右翻轉（像照鏡子）、調暗（像傍晚或陰天）、改顏色（像換了一台相機）、放大、移動（像車子開得比較近）、四張拼一張（把四張照片各縮小一半拼成一張，三角錐變小、搬到新的位置）。上下翻轉不做（路不會倒過來）。這些變化是隨機、同時發生的。照片翻過來，框也翻過來；照片放大，框也跟著放大、移動：框一定要跟著照片一起變，不然答案就錯了。30 張照片練 50 輪，模型一共看了 1,500 次照片，每一次看到的都不太一樣。就像同一個觀念，老師換很多種題目考你，你才是真的懂，不是背下某一題的答案。

對答案：位置錯多少，加上分數錯多少



scene-0061 #28（訓練照片）中間那個三角錐，第 **35** 輪：**IoU 0.90**、分數 **0.86**。白色虛線 = 標準答案，橘色框 = 模型猜的。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

模型猜完，就要對答案。錯的程度分兩種。位置錯多少：用 IoU，完全重疊是 1，完全沒碰到是 0，所以「1 − IoU」就是位置錯多少。分數錯多少：這裡明明有三角錐，答案是「有，百分之百」，模型給 0.11，就錯了 0.89；給 0.86，只錯 0.14。兩種錯加起來就是損失：位置愈準、分數愈對，損失愈小。（這是簡化的說法，YOLO11 實際用的公式比較複雜，框的部分用 CIoU 加 DFL，類別的部分用二元交叉熵，但意思一樣。）這個三角錐，第 1 輪 IoU 是 0.62、分數 0.11，到第 35 輪 IoU 0.90、分數 0.86。訓練不是每一輪都進步：第 9 輪框的位置還可以，分數卻掉到 0.01，到第 13 輪才又回來。

改參數：**260** 萬個旋鈕，一批與一輪

把模型想成一個超大的調音台，上面有 **2,604,883** 個旋鈕，這些旋鈕就是參數。每個旋鈕轉到哪裡，決定模型看到一張照片會猜出什麼框、給多少分。一開始旋鈕不是為三角錐調的，所以猜得很離譜。每次三步：猜一次、對答案算出錯多少，然後電腦算出每一個旋鈕往左轉還是往右轉會讓錯誤變少，

就把每個旋鈕都轉一點點。沒有人一個一個去調，往哪邊轉是電腦自己算的。第八週「大霧裡下山」的球，講的是同一件事：那時模型只有一個旋鈕（門檻），今天是 260 萬個旋鈕一起走下坡。

一批：電腦一次看 4 張照片，算一次錯多少。一輪：30 張全部看過一遍（epoch），所以一輪是 8 批。另外，這次的設定（Ultralytics 的預設）不是每一批都轉旋鈕，而是攢夠 64 張照片的錯誤才一起轉一次（叫梯度累積），所以 50 輪下來，旋鈕實際只轉了 27 次。每一次轉的量由學習率決定，都很小。

模型怎麼「只看一次」：19,320 個位置

照片 1600 × 900 縮到 1280 × 720，再補到 1280 × 736（模型需要 32 的倍數）。模型有三層格子，每格 8、16、32 個像素：160 × 92 = 14,720 個位置，80 × 46 = 3,680 個，40 × 23 = 920 個，加起來 14,720 + 3,680 + 920 = **19,320 個位置**。小的三角錐靠細的格子，大的靠粗的，像三種網眼大小的篩子：細的抓小石頭，粗的抓大石頭。這張測試照片分數 ≥ 0.25 的 89 個框裡，81 個來自最細那層，8 個來自中間那層，最粗那層 0 個。

每個位置猜的是整個框，不是只猜自己那一小格。格子只有 8 個像素寬，猜出來的框卻可以有 48 個像素寬，是格子的 6 倍。一個小位置怎麼知道框要多大？因為它看的不只是自己那 8 個像素：模型一層一層往上算，每個位置的結果參考了周圍一大片畫面。它輸出的是四個距離：從這個位置出發，往左、上、右、下，各到框的邊有多遠。位置加上四個距離，就是一個框。這四個距離為什麼會準，是練出來的。

訓練時才有答案。訓練時要決定：答案裡的這個三角錐，要拿哪幾個位置的猜測去跟它比？做法是：只看落在答案框裡面的位置，挑猜得最準的，最多 **10** 個，依分數、還有框跟答案重疊多少來挑。這幾個位置猜的框和分數，拿去跟答案對照，錯多少就改多少；其他所有位置，目標是分數 **0**：「這裡沒有三角錐」。這樣練出來，推論的時候不需要答案，三角錐上面的位置，自然就會給高分。也因為一個三角錐常常有好幾個位置給高分、猜出好幾個框，才需要刪重複那一步。

三種訓練條件：照片多、不縮太小

表 9　同樣 **17** 張測試照片、**90** 個三角錐

做法	訓練時間（**MacBook**）	框對	多框
原版 COCO 模型（沒教過三角錐）	—	0 / 90	0

做法	訓練時間（MacBook）	框對	多框
5 張照片，縮到 1280 寬	79 秒	32 / 90	587
30 張照片，縮到 640 寬	95 秒	34 / 90	362
30 張照片，縮到 1280 寬	160 秒	80 / 90	72

只用 5 張照片，它只學到「橘色的東西大概是」，所以看到橘色就框，多框了 587 個。30 張縮到 640 寬只框對 34 個：遠處的三角錐原本只有十幾個像素高，縮到 640 就只剩 5 個像素左右，連人都很難看出那是三角錐（第八週也講過：行人在 45 公尺外，縮到 640 以後只剩 19 個像素，就容易漏）。兩個結論：照片多一點有用；照片不要縮得太小，小東西才看得到。代價是時間：縮到 1280，練一次要 160 秒，比 640 多了快一倍。一台 MacBook 還算快；真正的公司用幾百萬張照片，要很多台電腦算好幾天。

混淆矩陣：把四種結果放進一張表

混淆矩陣是一張計數表：橫列、直欄都是類別，每一格數「模型說什麼、實際是什麼」的個數。兩個類別就是 2 × 2，N 個類別就是 N × N。叫「混淆」，是因為看哪一格的數字大，就看得出模型把什麼和什麼搞混。下面用第 2 節那張照片的數字（7、9、1）來填：

	實際上：有三角錐	實際上：沒有
模型說：有	TP 框對　7	FP 多框　9
模型說：沒有	FN 漏掉　1	TN 不算

精確度看橫的第一列：模型說「有」的一共 7 + 9 = 16 個，7 個是對的，7 ÷ 16 ≈ 44%。召回率看直的第一欄：實際上有的一共 7 + 1 = 8 個，找到 7 個，7 ÷ 8 ≈ 88%。偵測的 **TN** 沒有意義：照片上沒有三角錐的位置有無數個，每一個「沒框」都算 TN，數字大到沒有用。所以偵測只看 TP、FP、FN，也就是精確度和召回率，不用正確率（accuracy）：只要什麼都不框，正確率就接近 100%，毫無意義。看別人的混淆矩陣時，先看軸的標籤：有的書（以及這份講義）橫列是「預測」，scikit-learn 的橫列卻是「實際」。論文裡常看到的 **AP**、**mAP**（平均精確度），是把門檻從低拉到高，畫出精確度和召回率的曲線再取底下的面積，是這兩個分數的進階版。

九、術語表（中文· 越南文· English）

這一週每一張投影片最下面的「名詞條」列的，就是這張表裡的詞。「第一次在哪一節教」是指這個詞主要在哪一節出現。

中文	越南文	English	節
模型	Mô hình	model	第 1 節
三角錐	Cọc tiêu	traffic cone	第 1 節
訓練	Huấn luyện	training	第 1 節
推論	Suy luận	inference	第 1 節
評估	Đánh giá	evaluation	第 1 節
預訓練模型	Mô hình huấn luyện trước	pre-trained model	第 1 節
YOLO 只看一次	Chỉ nhìn một lần	You Only Look Once	第 1 節
標註	Gán nhãn	annotation	第 1 節
框	Khung giới hạn	bounding box	第 1 節
訓練組	Tập huấn luyện	training set	第 1 節
驗證組	Tập kiểm định	validation set	第 1 節
測試組	Tập kiểm tra	test set	第 1 節
資料洩漏	Rò rỉ dữ liệu	data leakage	第 1 節
遷移學習	Học chuyển giao	transfer learning	第 1 節
微調	Tinh chỉnh	fine-tuning	第 1 節
一輪	Một vòng	epoch	第 1 節
損失	Mất mát	loss	第 1 節
召回率	Độ bao phủ	recall	第 1 節
過度擬合	Quá khớp	overfitting	第 1 節
欠擬合	Thiếu khớp	underfitting	第 1 節
精確度	Độ chính xác	precision	第 2 節
信心分數	Điểm tin cậy	confidence score	第 2 節
刪重複（NMS）	Bỏ trùng	non-maximum suppression	第 2 節
IoU 重疊度	Tỷ lệ giao trên hợp	Intersection over Union	第 2 節
門檻	Ngưỡng	threshold	第 2 節

中文	越南文	English	節
多框	Thừa	false positive, FP	第 2 節
漏掉	Sót	false negative, FN	第 2 節
標準答案	Đáp án chuẩn	ground truth	第 2 節
漏標	Sót nhãn	missing label	第 2 節
資料引擎	Vòng lặp dữ liệu	data engine	第 2 節
YOLO 格式	Định dạng YOLO	YOLO format	第 2 節
正規化	Chuẩn hóa	normalization	第 2 節
資料增強	Tăng cường dữ liệu	data augmentation	第 2 節
拼圖（馬賽克）	Ghép ảnh mosaic	mosaic	第 2 節
網格	Ô lưới	grid cell	第 2 節
參數	Tham số	parameters	第 2 節
一批	Một lô	batch	第 2 節
解析度	Độ phân giải	resolution	第 2 節
推論速度	Tốc độ suy luận	inference speed	第 2 節
混淆矩陣	Ma trận nhầm lẫn	confusion matrix	第 2 節
真陰性	Âm tính thật	true negative, TN	第 2 節
偵測	Phát hiện	detection	第 3 節
追蹤	Theo dõi	tracking	第 3 節
追蹤編號	Số theo dõi	track ID	第 3 節
配對	Ghép cặp	association	第 3 節
速度	Tốc độ	speed	第 3 節
換號	Đổi số	ID switch	第 3 節
遮擋	Bị che khuất	occlusion	第 3 節
鳥瞰圖	Ảnh nhìn từ trên	bird's-eye view, BEV	第 3 節
佔據網格	Lưới chiếm chỗ	occupancy grid	第 3 節
預測	Dự đoán	prediction	第 3 節
規劃	Lập kế hoạch	planning	第 3 節
卡曼濾波	Bộ lọc Kalman	Kalman filter	第 3 節

十、資料來源與延伸影片

資料與工具

- 照片與標註：nuScenes mini（Motional），CC BY-NC-SA 4.0。Caesar et al., nuScenes: A multimodal dataset for autonomous driving, CVPR 2020。
- 模型：Ultralytics YOLO11n（AGPL-3.0）；預訓練資料 COCO（Lin et al., Microsoft COCO: Common Objects in Context, ECCV 2014）。YOLO 原始論文：Redmon et al., You Only Look Once: Unified, Real-Time Object Detection, CVPR 2016。

延伸影片（選用，不是必看）

影片	語言	時間	網址
NStatum〈Underfitting & Overfitting - Explained〉：練不夠與練過頭	英語動畫	0:00–2:52	youtu.be/o3DztvnfAJg
Levity〈What is Transfer Learning?〉：遷移學習	英語	0:00–2:45	youtu.be/vmjP6LjGaag
TED，Joseph Redmon〈How computers learn to recognize objects instantly〉：YOLO 只看一次	英語（有越南文字幕）	3:44–4:30	youtu.be/Cgxsv1riJhI?t=224
Data Dance〈Hiểu đúng về đánh giá mô hình ML〉：精確度與召回率	越南語動畫	2:51	youtu.be/8X3BMrDAQX0
StatQuest〈Machine Learning Fundamentals: The Confusion Matrix〉：混淆矩陣	英語	1:32–3:50	youtu.be/Kdsp6soqA7o?t=92
Datum Learning〈NMS explained in detail〉：刪重複	英語	0:08–4:10	youtu.be/v6lt0cGSHBI?t=8
DeepLearning.AI（吳恩達）〈Nonmax Suppression〉：刪重複	英語	2:03–3:45	youtu.be/VAo84c1hQX8?t=123
NVIDIA DRIVE Labs 第 5 集〈Tracking Objects With Surround Camera Vision〉	英語	1:26	youtu.be/aQwqD5cB2ck
Chalmers〈Challenges in Multi-Object Tracking〉：多目標追蹤的難處	英語	1:16–2:33	youtu.be/H5BjLdUcIoA?t=76
Yêu công nghệ〈Tracking gán id đối tượng python〉：越南路口，每台車一個號碼	無旁白	1:24	youtu.be/fkE-kbuL9iA